

Two Pointers

Today's Content:

- Pairs with $\text{sum} = k$.
- Pairs with $\text{difference} = k$.
- Subarrays with given sum
- Container with most water
- Important Announcement

Question: Given a sorted array and an integer k , find any pair (i, j) such that $A[i] + A[j] = k$ and $i \neq j$.

ex: $\text{arr} = [-5, -2, 1, 8, 10, 12, 15]$ $k = 11$

Brute Force Approach:

Take all pairs, check if $\text{sum} == k$.

$$T.C = O(N^2) \quad S.C = O(1)$$

Using Binary search:

<u>arr[i]</u>	<u>arr[j] = k - arr[i]</u>		<u>T.C</u>
-5	16	Check if 16 is present	$\log N$
-2	13		$\log N$
...			

$$T.C = O(N \log N) \quad S.C = O(1)$$

Using Hashmap:

- Dump elements in the hashmap.
- For every element, check if compair exist.

$$T.C = O(N) \quad S.C = O(N)$$

arr = [-5, -2, 1, 8, 10, 12, 15] k = 11

<u>A[i]</u>	<u>A[j]</u>	<u>Sum</u>	<u>Result</u>
-5	15	10 < 11	-5 can't form any pair.; i++
-2	15	13 > 11	j--
-2	12	10 < 11	-2 can't form any pair.; i++
1	12	13 > 11	j--
1	10	11 == 11	Return the pair.

```
func pairSumK (int[] arr, int k) {
```

```
i = 0; j = arr.length - 1;
```

```
while (i < j) {
```

if ($\text{arr}[i] + \text{arr}[j] == k$) return $[i, j]$;

```
else if (arr[i] + arr[j] < k) i++;
```

```
else j--;
```

1

```
return [];
```

T.C = $O(N)$ S.C = $O(1)$

Question: Count all the pairs whose sum = k in a distinct sorted array.

arr = [1, 2, 3, 4, 5, 6, 7, 8] k = 10

Diagram showing pointers i and j moving towards the center of the array. i points to 1, 2, 3 and j points to 7, 8.

<u>arr[i]</u>	<u>arr[j]</u>	<u>Sum</u>	<u>Result</u>
1	8	9 < 10	1 can't form any pair, i++
2	8	10 == 10	ans++, i++, j--
3	7	10 == 10	ans++, i++, j--
...			

```
func countPairs (int[] arr, int k) {
```

```
    i = 0; j = arr.length - 1;
```

```
    ans = 0;
```

```
    while (i < j) {
```

```
        sum = arr[i] + arr[j];
```

```
        if (sum == k) ans++; i++; j--;
```

```
        else if (sum < k) i++;
```

```
        else j--;
```

```
    }
```

```
    return ans; }
```

Question: Count all the pairs whose sum = k.

Note: Array can contain duplicates.

$$\begin{array}{ccccccc} & i & & i & & j & & j & & j & & j \\ & \downarrow & & \downarrow & & \swarrow & & \swarrow & & \swarrow & & \swarrow \\ \text{arr} = [& 2 & , & 3 & , & 3 & , & 10 & , & 10 & , & 10 & , & 15] \quad k = 13 \end{array}$$

O/P = 6

<u>arr[i]</u>	<u>arr[j]</u>	<u>sum</u>	<u>Result</u>	<u>[ans = 0]</u>
2	15	17	j--	
2	10	12	i++	
[1] 3	[5] 10	13	ans++ ; j--	ans = 1
3	[4] 10	13	ans++ ; j--	; ans = 2
3	[3] 10	13	ans++ ; j--	; ans = 3
[1] 3	[2] 3	6	i++	
[2] 3	[2] 3		⇒ Break	

$$\begin{array}{ccccccccccccccc} & i & & i & & i & & i & & i & & i & & j & & j & & j & & j & & j \\ & \downarrow & & \downarrow & & \downarrow & & \downarrow & & \downarrow & & \downarrow & & \swarrow & & \swarrow & & \swarrow & & \swarrow & & \swarrow \\ \text{arr} = [& 2 & , & 4 & , & 4 & , & 4 & , & 5 & , & 5 & , & 7 & , & 7 & , & 7 & , & 10 & , & 10 & , & 10 & , & 15] \end{array}$$

k = 14 ans = 0 12 count1 = count2 = count = 3

```
func countPairs (int[] arr, int k) {
```

```
    i = 0 ; j = arr.length - 1 ;
```

```
    ans = 0 ;
```

```
    while (i < j) {
```

```
        sum = arr[i] + arr[j] ;
```

```
        if (sum < k) i++ ;
```

```
        else if (sum > k) j-- ;
```

```
        else if (arr[i] == arr[j]) { // sum == k
```

```
            count = j - i + 1 ;
```

```
            ans += (count * (count - 1) / 2) ;
```

```
            break ;
```

```
        }
```

```
    else { // sum == k.
```

```
        count1 = 1 ;
```

```
        while (i < N - 1 && arr[i] == arr[i + 1]) {
```

```
            count1++ ;
```

```
            i++ ;
```

```
        }
```

```
        count2 = 1 ;
```

```

        while (j > 0 && arr[j] == arr[j-1]) {
            count2++;
            j--;
        }
        ans += count1 * count2; i++; j--;
    }
}
return ans;
}

```

T.C = $O(N)$ S.C = $O(1)$

Question: Given a sorted integer array and an integer k , find any pair (i, j) such that $A[j] - A[i] = k$ & $i \neq j$

ex: $arr = [-5, -2, 1, 8, 10, 12, 15]$ $k = 11$

Brute Force Approach

- Take all pairs and check their difference.

T.C = $O(N^2)$ S.C = $O(1)$.

Using Binary search

$$arr[j] - arr[i] = k$$

$$arr[j] = k + arr[i] \quad (\text{Co-pair})$$

- For every element, search for the co-pair.

T.C = $O(N \log N)$ S.C = $O(1)$

$i \searrow \quad i \searrow \quad \swarrow j \quad \swarrow j \quad \swarrow j \quad \swarrow j \quad \swarrow j$
 $arr = [-5, -2, 1, 8, 10, 12, 15]$ $k = 11$

<u>$arr[i]$</u>	<u>$arr[j]$</u>	<u>Diff</u>	<u>Result</u>
----------------------------	----------------------------	-------------	---------------

-5	15	$20 > 11$	$j--$
----	----	-----------	-------

-5	12	$17 > 11$	$j--$
----	----	-----------	-------

-5	10	$15 > 11$	$j--$
----	----	-----------	-------

-5	8	$13 > 11$	$j--$
----	---	-----------	-------

-5 1 6 < 11 i++

-2 1 3 < 11 i++

1 1 $\Rightarrow$ Break $\Rightarrow$ No pair found

Cur_Diff $\downarrow$ = arr[j] - arr[i] $\uparrow$ < k $\Rightarrow$ i++ $\Rightarrow$ FLAW

Cur_Diff $\downarrow$ = arr[j] $\downarrow$ - arr[i] > k $\Rightarrow$ j--

Cur_Sum $\uparrow$ = arr[i] $\uparrow$ + arr[j] < k $\Rightarrow$ i++

Cur_Sum $\downarrow$ = arr[i] + arr[j] $\downarrow$ > k $\Rightarrow$ j--

i $\downarrow$ j $\downarrow$

arr = [-5, -2, 1, 8, 10, 12, 15] k = 11

Cur_Diff = arr[j] - arr[i] < k $\Rightarrow$ Increase cur_diff.

Either arr[j] $\uparrow$ - arr[i] $\Rightarrow$ j++

or arr[j] - arr[i] $\downarrow \Rightarrow$ i-- $\Rightarrow$ Useless.

Cur_Diff = arr[j] - arr[i] > k $\Rightarrow$ Decrease cur_diff.

Either arr[j] $\downarrow$ - arr[i] $\Rightarrow$ j-- $\Rightarrow$ Useless.

or. arr[j] - arr[i] $\uparrow \Rightarrow$ i++

```

func diffPairK (int[] arr, int k) {
    i = 0; j = 1;
    while (i < j && j < N) {
        diff = arr[j] - arr[i]
        if (diff == k) return [i, j];
        else if (diff < k) j++;
        else {
            i++;
            if (j <= i) j = i + 1;
        }
    }
    return [];
}

```

T.C = $O(N)$ S.C = $O(1)$

$\begin{matrix} i & j \\ \downarrow & \downarrow \end{matrix}$
 arr = [-5, -2, 1, 8, 11, 12, 15] $k = 1$

<u>arr[i]</u>	<u>arr[j]</u>	<u>Diff</u>	<u>Result</u>
-5	-2	3 > 1	i++
-2	-2	⇒ Break.	

⇒ Count of pairs with Diff == k ⇒ TODO.

↳ Both with and without duplicates

Question: Given an integer array of +ve numbers and an integer k , check if there exists a subarray with $\text{sum} = k$.

ex: $\text{arr} = [1, 3, 15, 10, 20, 3, 23]$
 $k = 28$

Brute Force Approach

- Go through all subarray sum, check if $\text{sum} == k$.
↳ Carry Forward.

$$T.C = O(N^2) \quad S.C = O(1)$$

Using Two pointers:

$\text{arr} = [1, 3, 15, 10, 20, 3, 23]$
 $\text{PF} = [1, 4, 19, 29, 49, 52, 75]$

- Similar to $\text{Diff} == k$ on PF array.
- Also check if $\text{PF}[i] == k$

$$T.C = O(N) \quad S.C = O(N)$$

With optimised space:

arr = [1, 3, 15, 10, 20, 3, 23] k = 56

<u>arr[i]</u>	<u>arr[j]</u>	<u>PFSum</u>	<u>Result</u>	<u>new j</u> ↓
1	3	$3 < 56$	$j++$; $PFSum + arr[j]$	
	15	$18 < 56$	$j++$; $PFSum + arr[j]$	
	10	$28 < 56$	$j++$; $PFSum + arr[j]$	
	20	$48 < 56$	$j++$; $PFSum + arr[j]$	
	3	$51 < 56$	$j++$; $PFSum + arr[j]$	
	23	$74 > 56$	$i++$; $PFSum - arr[i] \leftarrow new j$	
3	23	$71 > 56$	$i++$; $PFSum - arr[i]$	
15	23	$56 == 56$	return	

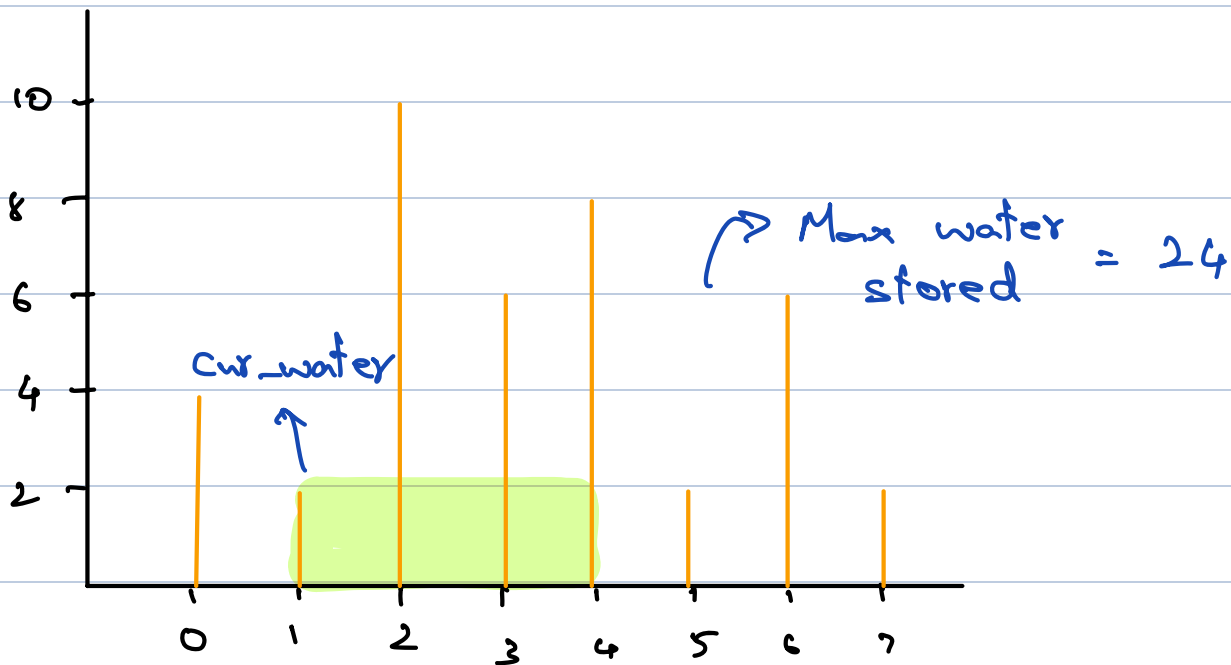
```

func subArraySum(int[] arr, int k) {
    i = 0 ; j = 1 ;
    pfsum = arr[j];
    while (i < j && j < N) {
        if (pfsum == k) return [i+1, j];
        else if (pfsum < k) {
            j++;
            pfsum += arr[j];
        }
        else {
            i++;
            if (j <= i) {
                j = i + 1;
                pfsum += arr[j];
            }
            pfsum -= arr[i];
        }
    }
    return [];
}

```

T.C = $O(N)$ S.C = $O(1)$.

Question: Find two walls that can form a container to store maximum water.



$arr = [4, 2, 10, 6, 8, 2, 6, 2]$

O/P = 24

$$\text{Water}[i, j] = \min(arr[i], arr[j]) * (j - i)$$

Brute Force Approach

- Check water stored in every pair.
- Take max(water)

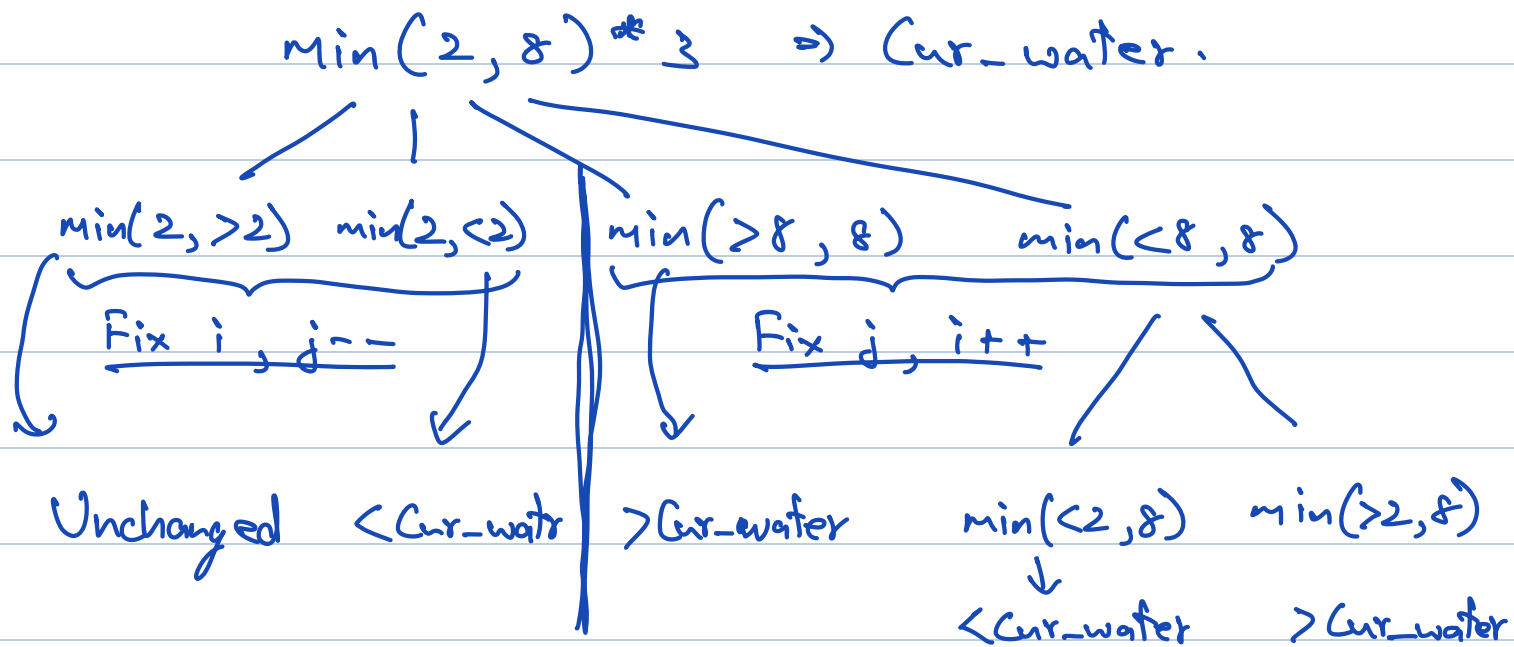
T.C = $O(N^2)$ S.C = $O(1)$

$$\text{Water} = \text{Min}(\text{height}_i, \text{height}_j) * \frac{\text{width}}{\downarrow \text{Fix}}$$

* Previously, we kept one parameter fixed and varied the other.

* Here, if we fix one parameter, then we can't change the other.

$$\text{if } \text{Water} = \text{Min}(\text{height}_i, \text{height}_j) * \text{width} \downarrow$$



Here, do $i++$.

In general, if $(arr[i] < arr[j]) \Rightarrow i++$
 else $j--$

```
func maxWaterContainer (int[] arr) {
```

```
    i = 0 ; j = arr.length.
```

```
    max_water = 0 ;
```

```
    while (i < j) {
```

```
        cur_water = Min(arr[i], arr[j]) * (j - i);
```

```
        max_water = max(max_water, cur_water);
```

```
        if (arr[i] < arr[j]) i++;
```

```
        else j--;
```

```
    }
```

```
    return max_water.
```

```
}
```

T.C = $O(N)$
S.C = $O(1)$

arr = [4, 2, 10, 6, 8, 2, 6, 2]

<u>arr[i]</u>	<u>arr[j]</u>	<u>cur_water</u>	<u>Result</u>	<u>max_water</u>
4	2	4	j--	4
4	6	24	i++	24
2	6	10	i++	24
10	6	24	j--	24
10	2	6	j--	24
10	8	16	j--	24
10	6	12	j--	24

Important Announcement

- My journey with you guys ends!
- New instructor joins on 24th.
- Classes from 29th.
- Contest - 5th August.